

Module 2

8051 Instruction Set: Addressing Modes, Data Transfer instructions, Arithmetic instructions, Logical instructions, Branch instructions, Bit manipulation instructions. Simple Assembly language program examples (without loops) to use these instructions

8051 data types and directives

The 8051 microcontroller has only one data type. It is 8 bits, and the size of each register is also 8 bits. It is the job of the programmer to break down data larger than 8 bits (00 to FFH, or 0 to 255 in decimal) to be processed by the CPU. The data types used by the 8051 can be positive or negative.

DB (define byte)

The DB directive is the most widely used data directive in the assembler. It is used to define the 8-bit data. When DB is used to define data, the numbers can be in decimal, binary, hex, or ASCII formats. For decimal, the “D” after the decimal number is optional, but using “B” (binary) and “H” (hexadecimal) for the others is required. Regardless of which is used, the assembler will convert the numbers into hex. To indicate ASCII, simply place the characters in quotation marks (‘like this’). The assembler will assign the ASCII code for the numbers or characters automatically. The DB directive is the only directive that can be used to define ASCII strings larger than two characters; therefore, it should be used for all ASCII data definitions. Following are some DB examples:

```
ORG 500H
DATA1: DB 28 ;DECIMAL(1C in hex)
DATA2: DB 00110101B ;BINARY (35 in hex)
DATA3: DB 39H ;HEX
ORG 510H
DATA4: DB "2591" ;ASCII NUMBERS
ORG 518H
DATA6: DB "My name is Joe" ;ASCII CHARACTERS
```

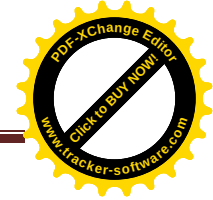
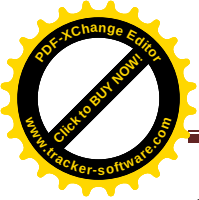
Either single or double quotes can be used around ASCII strings. This can be useful for strings, which contain a single quote such as “O’Leary”. DB is also used to allocate memory in byte-sized chunks.

Assembler directives

The following are some more widely used directives of the 8051.

ORG (origin)

The ORG directive is used to indicate the beginning of the address. The number that comes after ORG can be either in hex or in decimal. If the number is not



followed by H, it is decimal and the assembler will convert it to hex. Some assemblers use “. ORG” (notice the dot) instead of “ORG” for the origin directive.

EQU (equate)

This is used to define a constant without occupying a memory location. The EQU directive does not set aside storage for a data item but associates a constant value with a data label so that when the label appears in the program, its constant value will be substituted for the label. The following uses EQU for the counter constant and then the constant is used to load the R3 register.

```
COUNT    EQU    25
...
MOV      R3, #COUNT
```

When executing the instruction “MOV R3, #COUNT”, the register R3 will be loaded with the value 25 (notice the # sign). Assume that there is a constant (a fixed value) used in many different places in the program, and the programmer wants to change its value throughout. By the use of EQU, the programmer can change it once and the assembler will change all of its occurrences, rather than search the entire program trying to find every occurrence.

END directive

Another important pseudocode is the END directive. This indicates to the assembler the end of the source (asm) file. The END directive is the last line of an 8051 program, meaning that in the source code anything after the END directive is ignored by the assembler. Some assemblers use “. END” (notice the dot) instead of “END”.

ADDRESSING MODES OF 8051 :

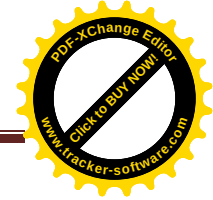
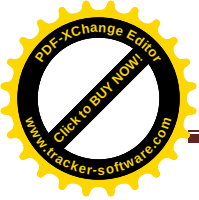
The way in which the data operands are accessed by different instructions is known as the addressing modes. There are various methods of denoting the data operands in the instruction. The 8051 microcontroller supports mainly 5 addressing modes. They are

1. Immediate addressing mode
2. Direct Addressing mode
3. Register addressing mode
4. Register Indirect addressing mode
5. Indexed addressing mode

Immediate addressing mode :

The addressing mode in which the source data operand is a constant and it is a part of the instruction itself is known as Immediate addressing mode. Normally the data must be preceded by a # sign. This addressing mode can be used to transfer the data into any of the registers including DPTR and I/O ports.

Ex: MOV A , # 27 H ; The data (constant) 27 is moved to the accumulator register



```
ADD R1 ,#45 H           ;Add the constant 45 to the contents of the
accumulator
MOV DPTR ,# 8245H ;Move the data 8245 into the data pointer register.
MOV P1, #88h           ; Move the data 88h to the port1
```

The EQU directive can be used to access immediate data as shown below

Ex:

```
Count EQU 30h
```

```
... ..
```

```
Mov R1, # Count      ; R1 = 30h
```

```
Mov DPTR, # Mydata   ; DPTR = 200h
```

```
... ..
```

```
ORG 200h
```

```
Mydata DB "INDIA"
```

Direct addressing mode:

The addressing mode in which the data operands are accessed from the internal RAM location (00 -7FH) and the address of the data operand is given in the instruction is known as direct addressing mode. The direct addressing mode most often uses the address range 30h-7Fh of Internal RAM and the SFRs

Ex:

```
MOV R1, 42H           ; Move the contents of RAM location 42h into R1
register
```

```
MOV 49H, A           ; Move the contents of the accumulator into the RAM
location 49.
```

```
ADD A, 56H           ; Add the contents of the RAM location 56h to the
accumulator
```

```
MOV 0E0h, R0         ; Move the data from the register R0 to the register A
```

Another major use of direct addressing mode is the data access from stack. Only direct addressing mode is allowed for pushing (writing) and popping (reading) the data into/from the stack.

Ex

```
PUSH 03h             ; Push the contents of the address 03h to the stack
memory
```

```
PUSH A               ; Invalid
```

Register addressing mode:

The addressing mode in which registers are used to hold the data operand to be manipulated is known as register addressing mode. Source and destination registers should match in size.

Ex:

```
MOV A,R0             ; Move the contents of the register R0 to the
accumulator
```

```
ADD A,R6             ;Add the contents of R6 register to the accumulator
```

```
MOV P1, R2           ;Move the contents of the R2 register into port 1
```



MOV R1, DPL ; Move the contents of DPL to register R1
MOV R5, R2 : This is invalid .The data transfer between the registers is not allowed.

Register Indirect addressing mode :

The addressing mode in which a register is used as a pointer to the data memory block is known as Register indirect addressing mode. Only registers R0 and R1 are used as pointers and they must be preceded by @ sign.

MOV A,@ R0 ;Move the contents of RAM location whose address is in R0 into A (accumulator)
MOV @ R1 , B ;Move the contents of B into RAM location whose address is held by R1

One of the advantages of register indirect addressing mode is that it makes accessing the data more dynamic than static as in the case of direct addressing mode.

Indexed addressing mode :

This addressing mode is used in accessing the data elements of lookup table entries located in program ROM space of 8051.

Ex : MOVC A,@ A+DPTR

The 16-bit register DPTR and register A are used to form the address of the data element stored in on-chip ROM. Here C denotes to code memory .In this instruction the contents of A are added to the 16-bit DPTR register to form the 16-bit address of the data operand.

8051 Instruction set

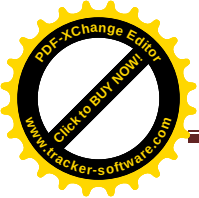
The instructions of 8051 can be broadly classified under the following headings.

1. Data transfer instructions
2. Arithmetic instructions
3. Logical instructions
4. Branch instructions
5. Subroutine instructions
6. Bit manipulation instructions

Instruction format

Label: OPCODE operands ;comment

Label : (*THIS IS NOT NECESSARY UNLESS THAT SPECIFIC LINE HAS TO BE ADDRESSED*). The label is a symbolic address for the instruction. When the program is assembled, the label will be given specific address in which that instruction is stored. Unless that specific line of instruction is needed by a branching instruction in the program, it is not necessary to label that line.



OPCODE: Opcode is the symbolic representation of the operation. The assembler converts the opcode to a unique binary code (machine language).

Operand: While opcode specifies what operation to perform, operand specifies where to perform that action. The operand field generally contains the source and destination location of the data. In some cases only source or destination location will be available instead of both. The operand will be either address of the data, or data itself.

Comment: Always comment will begin with ; or // symbol. To improve the program quality, programmer may always use comments in the program.

Data Transfer Instructions

Instructions used to transfer the data from one location to another. 8051 instruction set supports 28 distinct mnemonics to transfer a data from source to destination. All the data transfer mnemonics are classified under three main types.

MOV destination, Source

PUSH Source & POP destination

XCH destination, Source

Data transfer instructions changes only the destination Register / Memory location content, but the source content remains unaffected.

MOV destination, source are used to copy the data from register, memory location and the data itself to register/ memory location. Different types of instructions to perform data transfer are

1. Move the contents of a register Rn to A

MOV A,R2 ; Copies the data from register R2 to register A
(i.e) if A=11h & R2=22h before execution,
then after execution A=22h & R2= 222h.

2. Move the contents of a register A to Rn

MOV R4,A ; Copies the data from register A to register R4
(i.e) if A=11h & R4=22h before execution,
then after execution A=11h & R4= 11h.

3. Move an immediate 8 bit data to register A or to Rn or to a memory location(direct or indirect)

MOV A, #45h ;Copies the data 45h to Register A
MOV R1, #51h ;Copies the data 51h to Register R1
MOV 40h, #72h ;Copies the data 72h to memory address 40h
MOV DPTR, #1234h ;copies the 16bit data 1234h to Register DPTR

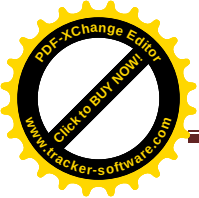
4. Move the contents of a memory location to A or A to a memory location using direct and indirect addressing.

i. MOV A, 65H ;Copies the data from memory location 65h to Register A

ii. MOV A, @R0 ;Copies the data from memory location pointed by register R0 to Register A.

iii. MOV 45H, A ;Copies the data from Register A to memory location 45h

iv. MOV @R1, A ;Copies the data from register A to memory address pointed by Register R1



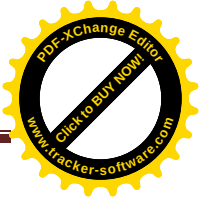
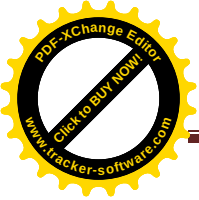
5. Move the contents of a memory location to Rn or Rn to a memory location using direct addressing
 - i. MOV R3, 65H ;Copies the data from memory location 65h to Register R3
 - ii. MOV 45H, R2 ;Copies the data from Register R2 to memory location 45h
6. Move the contents of memory location to another memory location using direct and indirect addressing
 - i. MOV 47H, 65H
 - ii. MOV 45H, @R0
7. Move the contents of an external memory to A or A to an external memory
 - i. MOVX A, @R1
 - ii. MOVX @R0, A
8. Move the contents of program memory to A
 - i. MOVC A, @A+PC
 - ii. MOVC A, @A+DPTR
9. Push and Pop instruction

	[SP]=07 //CONTENT OF SP IS 07 (DEFAULT VALUE)
MOV R6, #25H	[R6]=25H //CONTENT OF R6 IS 25H
	[R1]=12H //CONTENT OF R1 IS 12H
	[R4]=F3H //CONTENT OF R4 IS F3H
MOV R1, #12H	
MOV R4, #0F3H	
PUSH 6	[SP]=08 [08h]=[06h]=25H //CONTENT OF 08 IS 25H
	[SP]=09 [09]=[01]=12H //CONTENT OF 09 IS 12H
PUSH 1	[SP]=0A [0A]=[04]=F3H //CONTENT OF 0A IS F3H
PUSH 4	
POP 6	[06]=[0A]=F3H [SP]=09 //CONTENT OF 06 IS F3H
	[01]=[09]=12H [SP]=08 //CONTENT OF 01 IS 12H
	[04]=[08]=25H [SP]=07 //CONTENT OF 04 IS 25H
POP 1	
POP 4	

10. Exchange instructions

The content of source ie., register, direct memory or indirect memory will be exchanged with the contents of destination ie., accumulator.

- i. XCH A,R3 ; Exchanges the content of register R3 with Accumulator.



ii. XCH A,@R1

iii. XCH A,54h

11. **Exchange digit.** Exchange the lower order nibble of Accumulator with lower order nibble of the internal RAM location which is indirectly addressed by the register.

i. XCHD A,@R1

ii. XCHD A,@R0

Arithmetic Instructions

The 8051 can perform addition, subtraction. Multiplication and division operations on 8 bit numbers.

Addition

In this group, we have instructions to

- i. Add the contents of A with immediate data with or without carry.
 - i. ADD A, #45H ; Add the contents of Accumulator with the data 45h
 - ii. ADDC A, #0B4H ; Add the contents of Accumulator with the data B4h with carry flag
- ii. Add the contents of A with register Rn with or without carry.
 - i. ADD A, R5 ;Add the contents of Accumulator with the data in register R5
 - ii. ADDC A, R2 ;Add the contents of Accumulator with the data in register R2 and carry flag.
- iii. Add the contents of A with contents of memory with or without carry using direct and indirect addressing
 - i. ADD A, 51H ;Add the contents of Accumulator with the data in memory location 51h
 - ii. ADDC A, 75H ;Add the contents of Accumulator with the data in memory location 75h and carry flag.
- iii. ADD A, @R1 ; Add the contents of Accumulator with the data in memory location pointed by register R1
- iv. ADDC A, @R0 ; ; Add the contents of Accumulator with the data in memory location pointed by register R0 and the carry flag.

CY,AC and OV flags will be affected by this operation.

Subtraction

In this group, we have instructions to

- i. Subtract the immediate data from the contents of A with carry.
 - i. SUBB A, #45H ;Subtract the data 45h and the carry flag from the data in Accumulator
 - ii. SUBB A, #0B4H
- ii. Subtract the data in register Rn from contents of A with carry.
 - i. SUBB A, R5 ;Subtract the data in register R5 and the carry flag from the data in Accumulator

- ii. SUBB A, R2 ; Subtract the contents of A with contents of memory with or without carry using direct and indirect addressing
- iii. SUBB A, 51H ; Subtract the data in memory location 51h and the carry flag from the data in Accumulator
- iv. SUBB A, 75H
- v. SUBB A, @R1;
- vi. SUBB A, @R0 ; Subtract the data in memory location pointed by R0 and the carry flag from the data in Accumulator

CY AC and OV flags will be affected by this operation.

Multiplication

MUL AB. This instruction multiplies two 8 bit unsigned numbers which are stored in A and B register. After multiplication the lower byte of the result will be stored in accumulator and higher byte of result will be stored in B register.

MOV A,#0FFH ;[A]=FFH

MOV B,#0FFH ;[B]=FFH

MUL AB ;[A] x [B] = FF x FF = FE01 ;[A]=01H, [B]=FEH

Division

DIV AB. This instruction divides the 8 bit unsigned number which is stored in A by the 8 bit unsigned number which is stored in B register. After division the result will be stored in accumulator and remainder will be stored in B register.

Eg. MOV A, #0FFh ; [A] = FFh

MOV B,#02h ;[B]=02h

MULAB ;[A]/[B] = FFh/02h; [A] = 7Fh, [B] = 01h

Logical instructions

Logical AND

ANL destination, source: ANL does a bitwise "AND" operation between *source* and *destination*, leaving the resulting value in *destination*. The value in source is not affected. "AND" instruction logically AND the bits of source and destination.

ANL A,#DATA ANL A, Rn

ANL A,DIRECT ANL A,@Ri

ANL DIRECT,A ANL DIRECT, #DATA

Logical OR

ORL destination, source :ORL does a bitwise "OR" operation between *source* and *destination*,leaving the resulting value in *destination*. The value in source is not affected. " OR " instruction logically OR the bits of source and destination.

ORL A,#DATA

ORL A, Rn

ORL A,DIRECT

ORL A,@Ri

ORL DIRECT,A

ORL DIRECT, #DATA

Logical Ex-OR

XRL destination, source: XRL does a bitwise "EX-OR" operation between *source* and *destination*, leaving the resulting value in *destination*. The value in source is not affected. " XRL " instruction logically EX-OR the bits of source and destination.

XRL A,#DATA
XRL A,DIRECT

XRL A,Rn
XRL A,@Ri

XRL DIRECT,A

XRL DIRECT, #DATA

Logical NOT

CPL complements *operand*, leaving the result in *operand*. If *operand* is a single bit then the state of the bit will be reversed. If *operand* is the Accumulator then all the bits in the Accumulator will be reversed.

(i)**CPL A**, (ii)**CPL C**, (iii)**CPL bit address**

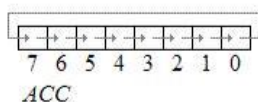
SWAP A – Exchange the upper nibble and lower nibble of Accumulator.

If [A] = 45h; After Execution [A] = 54h.

Rotate Instructions

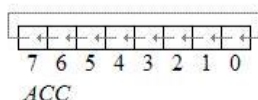
RR A

This instruction is rotate right the accumulator. Its operation is illustrated below. Each bit is shifted one location to the right, with bit 0 going to bit 7.



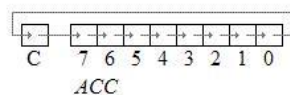
RL A

Rotate left the accumulator. Each bit is shifted one location to the left, with bit 7 going to bit 0



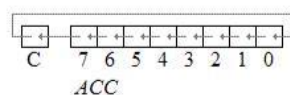
RRC A

Rotate right through the carry. Each bit is shifted one location to the right, with bit 0 going into the carry bit in the PSW, while the carry was at goes into bit 7



RLC A

Rotate left through the carry. Each bit is shifted one location to the left, with bit 7 going into the carry bit in the PSW, while the carry goes into bit 0.



Branch or Jump Instructions

These Instructions change the sequence of program execution from one memory address to another memory address by changing the contents of the register program counter.

There are 2 types of branch or jump instructions. They are:-

- (i) Conditional Jump
- (ii) Unconditional Jump instructions

Unconditional Jump instructions are of three kinds based on the range of Jump

- (i) Simple short Jump
- (ii) Absolute short Jump

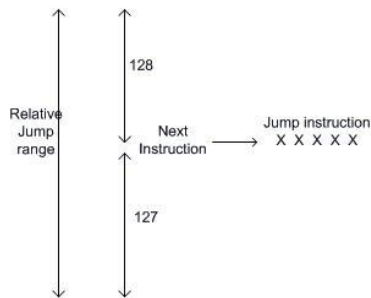
(iii) Long Jump

Range of Jump = Branch memory address – Current content of program counter.

Simple Short Jump or Relative Jump

Jump that replaces the PC (program counter) content with a new address that is in the range of +127 to -128 from the address of the next instruction following the Jump instruction is called a relative jump. Simple short jump and all conditional jumps are relative jumps. Relative jump instructions are of two bytes.

Schematically, the relative jump can be shown as follows: -



The advantages of the relative jump are as follows:-

1. Only 1 byte of jump address needs to be specified in the 2's complement form, ie. For jumping ahead, the range is 0 to 127 and for jumping back, the range is -1 to -128.
2. Specifying only one byte reduces the size of the instruction and speeds up program execution.
3. The program with relative jumps can be relocated without reassembling to generate absolute jump addresses.

Disadvantages of the relative jump: -

1. Short jump range ((i.e.)-128 to 127 from the instruction following the jump instruction) Instructions that use Relative Jump

In case of relative jump, the branch address is calculated by adding the second byte of the instruction with the current content of program counter(PC).

SJMP <relative address>; this is unconditional jump. The remaining relative jumps are conditional jumps

JC <relative address>

JNC <relative address>

JB bit, <relative address>

JNB bit, <relative address>

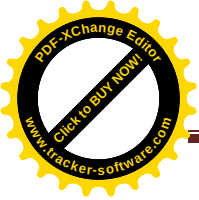
JBC bit, <relative address>

CJNE <destination byte>, <source byte>, <relative address>

DJNZ <byte>, <relative address>

JZ <relative address>

JNZ <relative address>



Short Absolute Jump

Short absolute jump instruction is also a 2 byte instruction. This AJMP instruction can be used when the jump address is located in the same page of the memory in the page in which the instruction AJMP is located.

In 8051, 64 kbyte of program memory space is divided into 32 pages of 2 kbyte each. The hexadecimal addresses of the pages are given as follows:-

Page (Hex)	Address (Hex)
00	0000 - 07FF
01	0800 - 0FFF
02	1000 - 17FF
3	1800 - 1FFF
.	
.	
1E	F000 - F7FF
1F	F800 - FFFF

It can be seen that the upper 5bits of all the addresses in a page remain same and hold the page number and the lower 11bits of all the addresses vary from all 0's to all 1's(i.e) 000h-7FFh. Branch address of the AJMP instruction can be calculated in the following manner. Upper 5 bits of the branch address is taken from the upper 5 bytes of the PC and the remaining 11 bits are obtained by concatenating the upper 3 bits of first byte of instruction with the second byte of instruction to form the 16bit branch address.

Advantage: The instruction length becomes 2 bytes.

Example of short absolute jump: -

ACALL <address 11>

AJMP <address 11>

Long Absolute Jump/Call

Applications that need to access anywhere in the entire program memory from 0000H to FFFFH use long absolute jump. Since the absolute address has to be specified as a part of the op-code, the instruction length is 3 bytes (except for JMP @ A+DPTR). This jump is not re-locatable.

Example: -

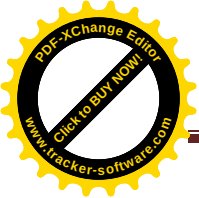
LCALL <address 16>

LJMP <address 16>

JMP @A+DPTR

Another classification of jump instructions is

1. Unconditional Jump
2. Conditional Jump



1. **The unconditional jump** is a jump in which control is transferred unconditionally to the target location.
 - a. **LJMP** (long jump). This is a 3-byte instruction. First byte is the op-code and second and third bytes represent the 16-bit target address which is any memory location from 0000 to FFFFH **eg: LJMP 3000H**
 - b. **AJMP**: this causes unconditional branch to the indicated address, by loading the 11 bit address to 0 -10 bits of the program counter. The destination must be therefore within the same 2K blocks.
 - c. **SJMP** (short jump). This is a 2-byte instruction. First byte is the op-code and second byte is the relative target address, 00 to FFH (forward +127 and backward -128 bytes from the current PC value). To calculate the target address of a short jump, the second byte is added to the PC value which is address of the instruction immediately below the jump.

2. Conditional Jump instructions.

JBC	Jump if bit = 1 and clear bit
JNB	Jump if bit = 0
JB	Jump if bit = 1
JNC	Jump if CY = 0
JC	Jump if CY = 1
CJNE reg,#data	Jump if [reg] ≠ #data
CJNE A,byte address	Jump if [A] ≠ [byte address]
DJNZ	Decrement and Jump if A ≠ 0
JNZ	Jump if A ≠ 0
JZ	Jump if A = 0

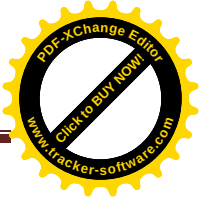
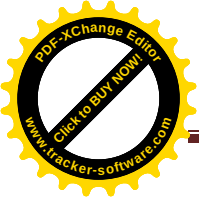
All conditional jumps are short jumps.

Bit level jump instructions:

Bit level JUMP instructions will check the conditions of the bit and if condition is true, it jumps to the address specified in the instruction. All the bit jumps are relative jumps.

JB bit, rel ; Jump to the relative address specified if direct bit in the bit address is set.

JNB bit, rel ; Jump to the relative address specified if direct bit in the bit address is clear.



JBC bit, rel ;Jump to the relative address specified if direct bit in the bit address is set

and then clear the bit

Subroutine CALL And RETURN Instructions

Subroutines are handled by CALL and RET instructions

There are two types of CALL instructions

1. LCALL address(16 bit)

This is long call instruction which unconditionally calls the subroutine located at the indicated 16 bit address. This is a 3 byte instruction. The LCALL instruction works as follows.

- a. During execution of LCALL, $[PC] = [PC] + 3$; (if address where LCALL resides is say, 0x3254; during execution of this instruction $[PC] = 3254h + 3h = 3257h$)
- b. $[SP] = [SP] + 1$; (if SP contains default value 07, then SP increments and $[SP] = 08$)
- c. $[[SP]] = [PC_{7-0}]$; (lower byte of PC content ie., 57 will be stored in memory location 08.)
- d. $[SP] = [SP] + 1$; (SP increments again and $[SP] = 09$)
- e. $[[SP]] = [PC_{15-8}]$; (higher byte of PC content ie., 32 will be stored in memory location 09.)

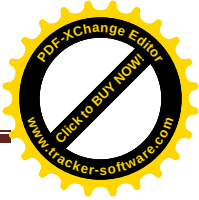
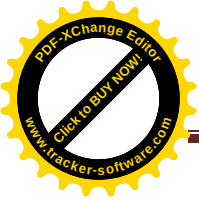
With these the address (0x3254) which was in PC is stored in stack.

- f. $[PC] = \text{address (16 bit)}$; the new address of subroutine is loaded to PC. No flags are affected.

2. ACALL address(11 bit)

This is absolute call instruction which unconditionally calls the subroutine located at the indicated 11 bit address. This is a 2 byte instruction. The ACALL instruction works as follows.

- a. During execution of ACALL, $[PC] = [PC] + 2$; (if address where ACALL resides is say, 0x8549; during execution of this instruction $[PC] = 8549h + 2h = 854Bh$)
- b. $[SP] = [SP] + 1$; (if SP contains default value 07, then SP increments and $[SP] = 08$)
- c. $[[SP]] = [PC_{7-0}]$; (lower byte of PC content ie., 4B will be stored in memory location 08.)



- d. $[SP] = [SP] + 1$; (SP increments again and $[SP] = 09$)
- e. $[[SP]] = [PC_{15-8}]$; (higher byte of PC content ie., 85 will be stored in memory location 09.
With these the address (0x854B) which was in PC is stored in stack.
- f. Lower 11 bits of the program counter $[PC_{10-0}]$ is loaded with a value concatenating the upper 3 bits of the first byte of the instruction with the second byte of the instruction

RET instruction

RET instruction pops top two contents from the stack and load it to PC.

$[PC_{15-8}] = [[SP]]$;content of current top of the stack will be moved to higher byte of PC.

$[SP] = [SP] - 1$; (SP decrements)

$[PC_{7-0}] = [[SP]]$;content of bottom of the stack will be moved to lower byte of PC.

$[SP] = [SP] - 1$; (SP decrements again)

Bit manipulation instructions.

8051 has 128 bit addressable memory. Bit addressable SFRs and bit addressable PORT pins. It is possible to perform following bit wise operations for these bit addressable locations.

1. LOGICAL AND

- a. $ANL\ C, BIT(BIT\ ADDRESS)$; 'logically AND' carry flag and content of bit address, store result in carry flag
- b. $ANL\ C, /BIT$; 'logically AND' carry flag and complement of content of bit address, store result in carry flag

2. LOGICAL OR

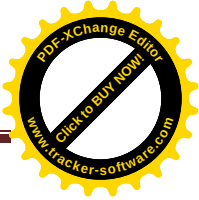
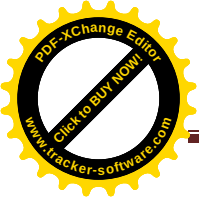
- a. $ORL\ C, BIT(BIT\ ADDRESS)$; 'logically OR' carry flag and content of bit address, store result in carry flag
- b. $ORL\ C, /BIT$; ; 'logically OR' carry flag and complement of content of bit address, store result in carry flag

3. CLR bit

- a. $CLR\ bit$; content of bit address specified will be cleared.
- b. $CLR\ C$; content of carry flag will be cleared.

4. CPL bit

- a. $CPL\ bit$; content of bit address specified will be complemented.
- b. $CPL\ C$; content of carry flag will be complemented.

**Example programs**

Write an ALP to copy the data 66h to the memory addresses starting from 30h to 34h

(i) Using direct addressing without using loop

(ii) Using indirect addressing without using Loop.

Answer

(i) Using direct addressing without using loop

```
ORG 00h
MOV 30h, #66h
MOV 31h, 30h
MOV 32h, 30h
MOV 33h, 30h
MOV 34h, 30h
L1:  SJMP L1
      END
```

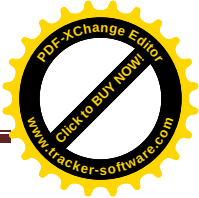
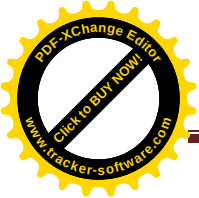
(ii) Using indirect addressing without using Loop.

```
ORG 00h
MOV R0, #30h
MOV A, #66h
MOV @R0, A
INC R0
MOV @R0, A
INC R0
MOV @R0, A
INC R0
MOV @R0, A
INC R0
MOV @R0, A
INC R0
MOV @R0, A
L1:  SJMP L1
      END
```

Write an ALP that checks the content of external RAM location 1000h is positive or negative. If Negative store the number itself in internal RAM location 50h else if Positive store the 2's complement of number in internal RAM location 50h.

Answer

```
ORG 00h
MOV DPTR, #1000h
MOVX A, @DPTR
RLC A
JC L1
CPL A
ADD A, #01h
L1:  MOV 50h,A
```



```
L2:  SJMP L2  
      END
```

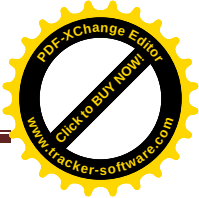
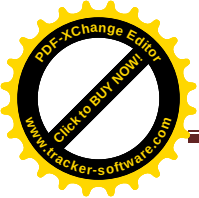
Write an ALP that checks the content of RAM location 50h is positive or negative. If positive store 00h else store FFh in RAM Location 80h.

```
      ORG 00h  
      MOV A, 50h  
      RLC A  
      JC L1    // If negative jump to location L1 to store FFh  
      MOV A, #00h // else store 00h  
      SJMP L2  
L1:   MOV A, #0FFh  
L2:   MOV 80h, A  
Stop: SJMP stop  
      END
```

Write an ALP that checks the content of RAM location 50h is ODD number or EVEN number. If EVEN store 00h else store FFh in RAM Location 80h.

```
      ORG 00h  
      MOV A, 50h  
      RRC A  
      JC L1    // If ODD jump to location L1 to store FFh  
      MOV A, #00h // else store 00h  
      SJMP L2  
L1:   MOV A, #0FFh  
L2:   MOV 80h, A  
Stop: SJMP stop  
      END
```

Write an ALP to convert a 2 digit decimal number in location 40h to its equivalent hex number and store in consecutive memory location

**Program:**

```
ORG      0000H
SJMP     30h
ORG      30h

MOV R0,#40H           //2-digit decimal number to be converted is given in data
                    memory 40h
MOV A,@R0
ANL A,#0F0H           //obtain upper decimal digit
SWAP A                //bring to the units place
MOV B,#0AH            //MULTIPLY tens digit with #0A-to get tens in hex
MUL AB
MOV R1,A              //temporarily store the converted tens value
MOV A,@R0              //get the decimal number again
ANL A,#0FH            //obtain the units digit
ADD A,R1              //add to the converted tens value
INC R0                //increment data address to store the result
MOV @R0,A             //converted hexadecimal number in next location
HERE:SJMP HERE

END
```

Write an ALP to convert a 2 digit Hexadecimal number in location 40h to its equalent decimal number and store the result in memory location 50h & 51h

Program:

```
ORG      0000H
SJMP     30h
ORG      30h
MOV R0,#40H
MOV A,@R0              //Get hex number
    11:MOV B,#10
    DIV AB              //divide by 10 (0AH)
    INC R0
    MOV @R0,B           //Store the remainder (in B) In units place
    JNZ 11              //Divide the quotient in A by 10
    MOV A,42H
    SWAP A
    ORL A,41H
    MOV 51h,A
    MOV 50H,43H
    HERE:SJMP HERE
END
```

